

Marking guide

General Questions (14 marks)

1. What is a design pattern? (2 marks)

A design pattern is a solution to a general problem that software developers faced during software development. These solutions were obtained by trial and error by numerous software developers over quite a substantial period of time.

or Solution to a recurring problem in context amid competing concerns. It tries to bring together and document the core solution to a given problem.

or A design pattern names, abstracts, and identifies the key aspects of a common design structure that make it useful for creating a reusable object-oriented design.

2. The 23 design patterns in the GoF book are categorized by purpose. Outline each purpose explaining what aspect of software engineering each purpose tries to address. (6 marks)



One mark for the category and one mark for the description.

- **Creational Patterns:** These patterns abstract the instantiation process. They help make a system independent of how its objects are created, composed, and represented.
- **Structural Patterns:** These patterns are concerned with how classes and objects are composed to form larger structures.
- **Behavioral Patterns:** These patterns are concerned with algorithms and the assignment of responsibilities between objects.

3. Explain 3 benefits of using design patterns in a software development project. (6 marks)



One mark for each benefit and one mark for its description.

- **Communication and Collaboration:** Design patterns serve as a common vocabulary for developers to communicate and collaborate effectively. Using patterns, team members can quickly understand and discuss design decisions, leading to better collaboration and faster development cycles.
- **Reusability:** Design patterns provide proven solutions to common problems in software design. By following established patterns, developers can reuse design and implementation solutions across different projects, saving time and effort.
- **Maintainability:** Using design patterns improves code maintainability by promoting clean, understandable, and modular code. Patterns encapsulate best practices and design principles, making it easier for developers to understand and modify code written by others.
- **Performance Optimization:** Certain design patterns can help optimize performance by promoting efficient algorithms, data structures, and code organization. For example, the Singleton pattern can reduce memory usage by ensuring that only one instance of a class is created.
- **Adaptability to Change:** Design patterns make software systems more adaptable to change by providing flexible and modular architectures. Patterns help separate concerns and dependencies, making it easier to modify and extend individual components without affecting the entire system.
- **Solving Common Problems:** Design patterns address recurring design problems in software development. They provide standardized solutions that have been thoroughly tested and proven effective in solving specific types of problems, such as object creation, communication between objects, or handling complex workflows.
- **Scalability:** Design patterns help create flexible and scalable software architectures. They provide guidelines for structuring code and organizing components, making it easier to accommodate changes and additions to the system as it grows in size and complexity.

Creational Patterns (10 marks)

1. **List the 5 creational design patterns cataloged in the GoF seminal book (2 marks)**
 - Singleton pattern

- Prototype pattern
- Factory method pattern
- Abstract factory pattern
- Builder pattern

2. **One of the creational patterns in the GoF book is the Singleton pattern. Outline 2 reasons why a given class would need to implement the singleton pattern. (2 marks)**

- **Resource Sharing:** Sometimes, you need to share a common resource or state across multiple parts of your application. By implementing a class as a singleton, you ensure that all components of your application access the same instance of that class, allowing for centralized management and sharing of resources.
- **Global Configuration:** Singleton classes are often used to manage global configuration settings or application state. For example, a configuration manager class might be implemented as a singleton to provide access to configuration settings from any part of the application.
- **Performance Optimization:** By creating only one instance of a class and reusing it throughout the application, you can potentially improve performance by avoiding the overhead of creating and destroying multiple instances of the same class.
- **Thread Safety:** Singleton classes can be designed to provide thread-safe access to shared resources, ensuring that multiple threads can safely interact with the singleton instance without causing data corruption or concurrency issues.
- **Lazy Initialization:** Singleton classes often use lazy initialization, meaning that the instance is created only when it's first requested. This can be useful for classes that are expensive to create or require significant initialization logic.

```
public class FileLogger
{
    private string fileName;
    public FileLogger(string fileName)
```

```

    {
        this.fileName = fileName;
    }
    public void Log(string message) {}
}

```

3. The code above shows an implementation of a `FileLogger` class. This class is used to write logs to a text file. We would want to implement the singleton pattern. Refactor this code to the singleton pattern. Explain your design decisions. (6 marks)

```

public class FileLogger
{
    private static FileLogger instance = null;
    private string filename;

    private FileLogger(string filename)
    {
        this.filename = filename;
    }

    public void Log(string message) {}

    public static FileLogger Create(string filename)
    {
        if(instance == null)
            instance = new FileLogger(filename);
        return instance;
    }
}

```

- We add a static private field whose type is the type of the class that we are creating. This field must be static since it will be accessed by a static method that creates the singleton.
- We make the class constructor private so that client classes can't access it to new up multiple instances.

- Finally we create a static method that is responsible for creating an instance of this class. It checks if the private field is null meaning the class hasn't been instantiated, creates a new instance, caches it so that all future invocations will use this instance.



This code won't work in a concurrent environment due to race conditions. In such cases, we use synchronization primitives to enforce mutual exclusion.

Structural Patterns (16 marks)

Consider a class that implements a `LinkedList` as shown below. We shall show a few operations for illustrative purposes.

```
public class LinkedList<T>
{
    public int Size();
    public void Add(T element);
    public T At(int index);
    public void Delete(int index);
    ...
}
```

Imagine that you want to implement a `Stack`. You may be forced to use inheritance to implement the `Stack` class as shown below.

```
public class Stack<T> inherits LinkedList<T>
{
    public void Push(T element){ Add(element); }
    public void Pop() { Delete(Size() - 1); }
    public T Top(){ return At(Size() - 1); }
}
```

1. What is the problem with this approach to using inheritance for code reuse? (2 marks)

The problem with this approach to code reuse is that it violates the Liskov substitution principle. Inheritance models an "is a" relationship. The code above

is trying to claim that a stack is a linked list which is not true. Nothing stops developers from writing such code

```
Stack<int> stack = new Stack<int>();
stack.Push(4);
stack.Push(44);
stack.Push(1);
stack.Push(6);
int secondElement = stack.At(1);
```

The last line violates the semantics of a stack. We can only access its head. On the other hand, we can access any item of a linked list. The stack class still exposes all the `LinkedList` methods it inherited and this leads to incorrect code.

2. Imagine that we have a class (client) that is expecting a `Stack` but we have access to the `LinkedList` class provided in a given library whose code we have no access to. Use the adapter pattern to adapt the `LinkedList` class to a `Stack`. (6 marks)

Let us imagine that the client is expecting the following stack interface.

```
public interface IStack<T>
{
    public void Push(T element);
    public void Pop();
    public T Top();
}
```

We implement an adapter class that wraps the existing `LinkedList` incompatible class and implements the interface using the methods of the wrapped class.



Note that the wrapped variable is private so that client code can't access its functionality. This approach to code reusability is known as **composition**.

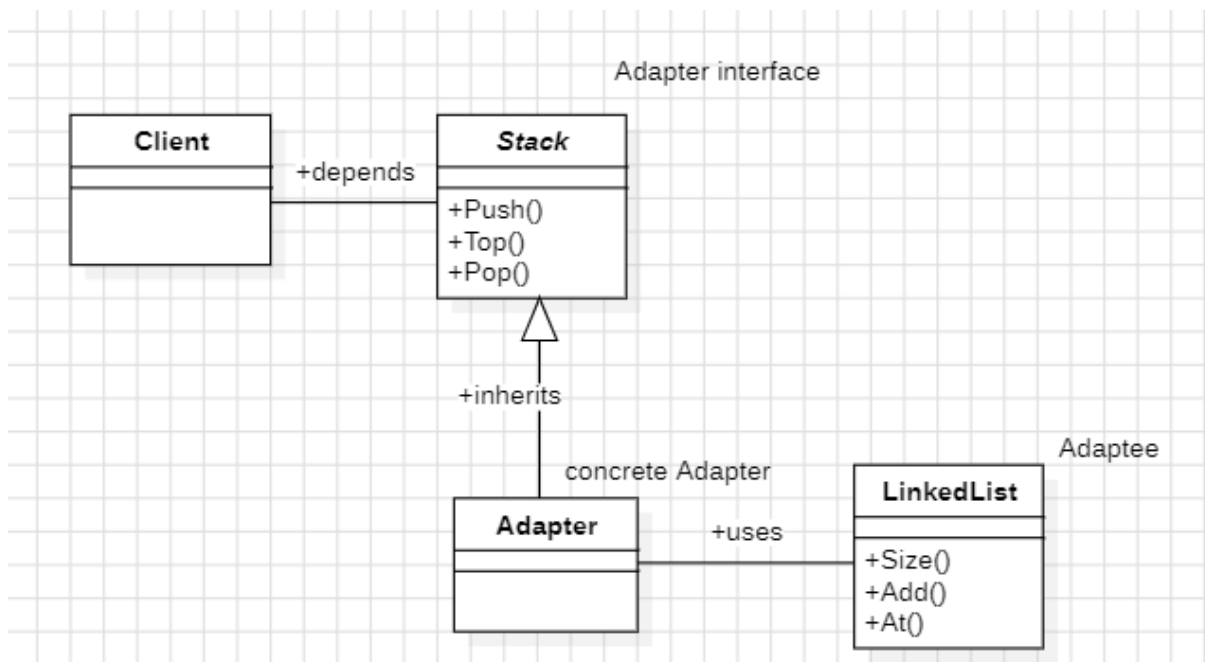
```

public class Adapter<T> implements IStack<T>
{
    private LinkedList list;
    public Adapter(LinkedList list){ this.list = list; }

    public void Push(T element){ list.Add(element); }
    public void Pop() { list.Delete(list.Size() - 1); }
    public T Top(){ return list.At(list.Size() - 1); }
}

```

3. Draw the UML diagram of the resulting design. Just focus on the classes association and not the methods. (2 marks)



4. Describe the facade pattern highlighting any two benefits of using it and any two applicable scenarios. (6 Marks)



2 marks for the description, 1 mark for each benefit and one mark for each application.

Description: The Facade patterns provides a unified interface to a set of interfaces in a subsystem. It defines a higher-level interface that makes the subsystem easier to use. This is because clients don't have to know a lot of details in order to use the subsystem. it also allows subsystem internal details to change without affecting clients.

Benefits

- It shields clients from subsystem components, thereby reducing the number of objects that clients deal with and making the subsystem easier to use.
- It promotes weak coupling between the subsystem and its clients.

Applicability

- You want to provide a simple interface to a complex subsystem.
- Layering of subsystems